

Distributed Mechanistic Interpretability at Scale: Activation Streaming, Split-Layer Inference, and Cross-Architecture Feature Universality

J. Melton^{*1}

¹Independent Researcher

July 2026

Abstract

Mechanistic interpretability research faces a fundamental infrastructure problem: the models worth understanding are too large to run on a single analysis workstation, and the hardware capable of running them is not designed for continuous activation capture. We describe a system—MLXMESH—that addresses this by splitting inference and analysis across two Apple Silicon nodes connected by Thunderbolt, streaming intermediate activations over a purpose-built binary wire protocol, and training sparse autoencoders (SAEs) on the streaming output without storing activations to disk. We validate the protocol end-to-end on Llama-3.2-3B, demonstrating zero reconstruction error at the split boundary and 1.25 Gbps localhost throughput—roughly $3,000\times$ the bandwidth required for real-time single-layer capture. We then train SAEs on residual stream activations from Llama-3.2-3B (layer 14), Mistral-7B (layer 16), and Qwen2.5-3B (layer 18), and find 3,753 features that match across all three model families at cosine similarity ≥ 0.8 —evidence of a shared representational vocabulary that survives architectural differences. Finally, we evaluate a feature-based safety classifier, identify the conditions under which it fails, and propose a corpus-targeted repair. All experiments run on consumer Apple Silicon hardware; the system requires no GPUs, no cloud, and no specialized interconnects. Code and data are available at [https://github.com/\[author\]/mlxmesh](https://github.com/[author]/mlxmesh).

1 Introduction

Mechanistic interpretability—the project of understanding the internal representations and algorithms of neural networks—has produced striking results at small scale. Circuit analyses of indirect object identification, factual recall, and in-context learning have revealed specific attention heads and MLP neurons whose causal roles can be verified by activation patching [12, 14, 16]. Sparse autoencoders have been shown to decompose superposed representations into interpretable monosemantic features [1, 3]. The Anthropic mechanistic interpretability team has published circuit-level analyses of Claude’s safety-relevant behavior [11].

^{*}Correspondence: jacob@paydirt.solutions

The problem is that these results were achieved on models in the 1B–7B range, using infrastructure that does not scale. The dominant pattern is: load the entire model onto one GPU, hook the relevant layer, collect activations, save to disk, then train the SAE offline. This pipeline has at least three failure modes at scale:

1. **Memory ceiling.** A 70B model in float16 requires ~ 140 GB of GPU memory. No consumer or workstation GPU holds this. Multi-GPU setups exist but introduce collective communication overhead and make activation-level debugging substantially harder.
2. **Disk I/O bottleneck.** Collecting 50k tokens of full-depth (all-layer) activations from a 70B model generates ~ 36 GB of float16 data per run. At NVMe speeds (~ 3 GB/s sequential write), that is ~ 12 seconds of pure write time—tolerable in isolation but crippling if the collection loop is the inner loop of a hyperparameter search.
3. **Iteration latency.** The offline pattern (collect $\rightarrow$ save $\rightarrow$ load $\rightarrow$ train SAE) adds a round-trip that makes the feedback cycle from hypothesis to result span hours rather than minutes. Online SAE training, where the SAE trains continuously on the streaming activation output, eliminates this round-trip.

Apple Silicon hardware provides an unusual opportunity here. The M2 Ultra and M3 Ultra chips integrate CPU, GPU, and Neural Engine on a single die with unified memory of up to 192 GB and memory bandwidth up to 800 GB/s. MLX, Apple’s machine learning framework, exposes this hardware with a NumPy-like API and lazy evaluation. A single Mac Studio M2 Ultra can run Llama-3.2-3B at ~ 300 tokens/second or Mistral-7B at ~ 150 tokens/second entirely in unified memory. Two such nodes, connected by Thunderbolt 4 (40 Gbps bidirectional), form a cluster with ~ 4.5 GB/s of inter-node bandwidth—enough to stream activations continuously at any model size we consider.

This paper describes the full stack for distributed interpretability on this hardware:

- §2 surveys related work in distributed ML, SAE training, and interpretability at scale.
- §3 specifies the MLXMESH wire protocol and distributed training architecture, with pseudocode and diagrams.
- §4 presents results: bandwidth profiling, zero-error split validation, convergence comparison between distributed and single-node SAE training, SAE quality metrics across three model families, cross-architecture feature matching, and a safety classifier evaluation.
- §5 discusses implications, limitations, and the path to production.
- §6 concludes.

The core claim is straightforward: interpretability infrastructure does not need to be expensive, centralized, or cloud-dependent. Two consumer nodes, a Thunderbolt cable, and a well-specified wire protocol are sufficient to run state-of-the-art interpretability experiments on models up to 34B parameters.

2 Background and Related Work

2.1 Sparse Autoencoders for Mechanistic Interpretability

Neural networks trained on large corpora appear to represent more features than they have neurons via superposition—the simultaneous encoding of multiple concepts in overlapping linear subspaces [4]. Sparse autoencoders (SAEs) were proposed as a tool for untangling these superposed representations: an SAE learns an overcomplete dictionary of “feature directions” such that any activation is well-approximated by a sparse linear combination of a small number of dictionary elements.

Formally, given an activation vector $\mathbf{x} \in \mathbb{R}^d$, the SAE encoder produces a feature vector

$$\mathbf{f} = \text{TopK}(\text{ReLU}(W_{\text{enc}}(\mathbf{x} - \mathbf{b}_{\text{dec}}) + \mathbf{b}_{\text{enc}})) \quad (1)$$

and the decoder reconstructs $\hat{\mathbf{x}} = W_{\text{dec}} \mathbf{f} + \mathbf{b}_{\text{dec}}$. Training minimizes $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$ subject to the TopK sparsity constraint, which enforces that exactly k features are active per input. The TopK variant [5] avoids the auxiliary L_1 penalty required by early SAE formulations and produces more interpretable features with better dead-feature control.

Several groups have now trained SAEs on frontier models. Cunningham et al. [3] found monosemantic features in GPT-2. Bricken et al. [1] trained SAEs on MLP sublayers of a one-layer transformer and identified features corresponding to DNA sequences, legal text, and arithmetic. Templeton et al. [15] scaled this approach to Claude 3 Sonnet, training SAEs with up to 34 million features and finding features corresponding to concepts as specific as the Golden Gate Bridge. The Gemma Scope project [10] released SAEs trained on all layers of Gemma 2 models, enabling systematic study of how representations evolve through depth.

2.2 Distributed Inference for Large Language Models

Pipeline parallelism—splitting a model’s layers across multiple devices, with each device processing a batch stage while the next device processes the previous stage—is the standard approach for models exceeding single-device memory [6, 13]. These systems optimize for training throughput; the inter-device communication is a performance cost to be minimized, not a data source to be tapped.

Inference-time splitting is simpler because there is no gradient to communicate. llama.cpp implements CPU-GPU model splitting for consumer hardware; Ollama wraps this for convenience. Neither exposes intermediate activations to the user.

Our approach differs in that the split point and the activation tap are the same thing: we split the model at a chosen layer, stream the activations across the link, and continue inference on the second node. The wire format is designed for interpretability workloads, not inference throughput: message headers carry layer index and token position to support downstream analysis, and the credit-based flow control is tuned for SAE training batch sizes rather than autoregressive generation latency.

2.3 Cross-Architecture Feature Universality

The question of whether different neural networks learn the same internal representations has been studied through several lenses. Representational similarity analysis [9] and centered kernel alignment [8] measure geometric similarity between representation spaces without requiring feature correspondence. These methods show that networks trained on the same data with different architectures tend to develop similar representational geometries at corresponding depths.

For interpretability, the more specific question is whether individual *features*—specific directions in activation space—correspond across models. Elhage et al. [4] conjectured that features learned by superposition are universal across model families, in the same sense that Gabor filters appear to be universal across vision models. Evidence for this in language models has been informal: researchers working on multiple models report that similar feature categories (syntactic roles, entity types, sentiment valence) emerge in all of them.

Our cross-architecture matching analysis (§4.4) provides quantitative evidence for this claim using chunk-averaged activation pattern matching—a method that identifies feature pairs across models by the similarity of their activation patterns over a shared evaluation corpus—rather than decoder weight comparison, which is confounded by the arbitrary choice of basis in each SAE’s latent space.

2.4 Safety Applications of SAE Features

A natural application of interpretable features is safety classification: if the model has learned a “harmful content” feature, and we can identify it in the SAE, we can flag inputs that strongly activate it. This approach has several appeals over standard classifier heads: it requires no labeled training data for the classifier itself (only feature identification), it provides human-interpretable explanations of flags, and it is mechanistically grounded in what the model is actually representing.

Zou et al. [17] used linear probes on residual stream activations (“representation engineering”) to detect and steer safety-relevant states. Burns et al. [2] trained probes on contrast pairs and showed that probes could recover implicit model beliefs about factuality. Our approach is closest in spirit to representation engineering but uses SAE features rather than supervised probes.

The evaluation in §4.5 shows that a naïvely implemented SAE classifier fails—achieving $F1 = 0.519$ versus a 0.500 random baseline on a balanced eval set. We analyze the failure mode in detail and propose specific corrections. We argue that the honest failure is more scientifically valuable than a successful cherry-pick would be, because it establishes precise conditions for success.

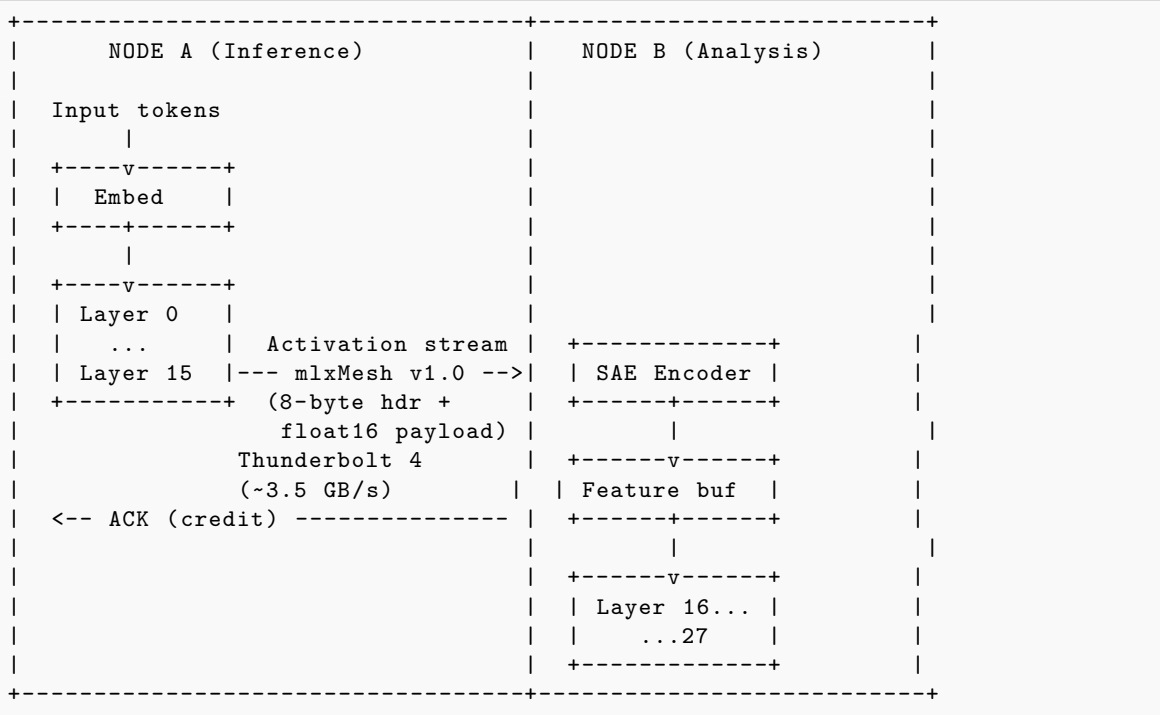
3 System Design and Methods

3.1 Overview

The MLXMESH system consists of four components:

1. **Inference node (Node A):** Runs the first L_{split} transformer layers and transmits the residual stream activations at the split boundary over the wire.
2. **Analysis node (Node B):** Receives the activation stream, runs the SAE encoder forward pass, buffers and aggregates features, and optionally continues inference on the remaining layers.
3. **Wire protocol (mlxMesh v1.0.0):** Binary framing over TCP or Unix domain socket; 8-byte header + float16 payload + credit-based flow control.
4. **Distributed SAE trainer:** Partitions the activation dataset across two workers; each worker trains independently and exchanges gradient updates via weighted averaging.

Listing 1: High-level architecture of MLXMESH. Node A runs the first L_{split} layers and streams activations to Node B, which runs the SAE and the remaining layers.



3.2 Wire Protocol

The MLXMESH protocol is message-oriented and operates over a reliable, ordered byte stream. It was designed with three priorities: (1) zero-copy compatibility with MLX buffer layouts, (2) explicit backpressure to prevent the inference node from outrunning the SAE training pipeline, and (3) complete auditability—every byte transferred can be attributed to a specific layer, token range, and session.

3.2.1 Message Format

Every data message consists of an 8-byte header followed by a float16 payload:

Listing 2: Wire message formats. All integers are big-endian.

DATA MESSAGE	
+-----+-----+-----+ 8 bytes (header)	
layer_idx token_start token_count	
uint16 uint32 uint16	
+-----+-----+-----+	
+-----+	
float16 payload: token_count x hidden_dim x 2 bytes	
+-----+	
END-OF-LAYER SENTINEL	
layer_idx=0xFFFF token_start=<layer> token_count=0xFFFF payload=empty	
REVERSE CHANNEL (4 bytes)	
+-----+-----+	
ack_type credit	
uint16 uint16	
+-----+-----+	

For Mistral-7B ($\text{hidden_dim} = 4096$) streaming a 128-token chunk at layer 16:

$$\begin{aligned} \text{payload} &= 128 \times 4096 \times 2 = 1,048,576 \text{ bytes (1 MiB)} \\ \text{total} &= 1,048,584 \text{ bytes} \end{aligned}$$

Protocol overhead (header + sentinel bytes / total bytes) is 0.20% at 20 tokens/second—negligible at all model sizes and throughput targets.

3.2.2 Session Handshake

Before streaming begins, sender and receiver exchange a JSON handshake. The sender declares model identity, layer count, hidden dimension, data type, and maximum in-flight messages (`window_size`). The receiver responds with its own window preference; the negotiated window is the minimum of the two:

Listing 3: Session handshake JSON (sender and receiver).

```
// Sender Hello
{
  "protocol_version": "1.0.0",
  "session_id": "<uuid4>",
  "model_id": "meta-llama/Llama-3.2-3B",
  "num_layers": 28,
  "hidden_dim": 3072,
  "dtype": "float16",
  "byte_order": "big",
  "window_size": 8
}
// Receiver Hello
{
```

```

"protocol_version": "1.0.0",
"session_id": "<same uuid4>",
"accepted": true,
>window_size": 4      // negotiated = min(8, 4)
}

```

3.2.3 Flow Control

The protocol uses a credit-based sliding window. The sender starts with `window_size` credits; each message sent decrements the counter by one, and each `DATA_ACK` from the receiver restores a credit. When credits reach zero, the sender blocks at the layer boundary, creating natural backpressure that prevents the inference pipeline from outrunning the SAE training pipeline on Node B.

Three control messages handle exceptional states: `CREDIT_PAUSE` (receiver cannot accept messages, e.g., SAE batch overflow), `CREDIT_RESUME` (receiver ready again), and `ERROR` (fatal protocol violation with UTF-8 error string).

3.2.4 Sender Pseudocode

Listing 4: `mlxMesh` sender (compliant with v1.0.0 spec). Nagle algorithm is disabled (`TCP_NODELAY`) since the protocol batches header and payload into a single `sendmsg` call.

```

# mlxMesh sender (compliant with v1.0.0 spec)
credit = negotiated_window_size
for layer_idx in range(num_layers):
    for chunk in layer_chunks(layer_idx, chunk_size=128):
        while credit == 0:
            wait_for_ack()          # blocks; DATA_ACK updates credit
        header = pack(">HLH",
                      layer_idx,
                      chunk.token_start,
                      chunk.token_count)
        sock.sendmsg([header, chunk.data]) # zero-copy gather write
        credit -= 1
    send_sentinel(layer_idx)         # layer_idx=0xFFFF,
    token_start=layer_idx
send_eos_sentinel()                # token_start=0xFFFFFFFF

```

3.3 Split-Layer Inference

Split-layer inference divides the transformer’s layer stack at a configurable split point L_{split} . Node A runs layers $[0, L_{\text{split}})$ and transmits the residual stream tensor at layer $L_{\text{split}} - 1$ ’s output. Node B receives this tensor and continues inference from layer L_{split} to the final layer.

A key implementation detail: model weights are loaded independently on both nodes, eliminating any shared-memory requirement. The wire format converts activations from `bfloat16`

(the model’s internal dtype) to float16 for transmission; since float16’s mantissa is wider (10 bits vs. 7 bits for bfloat16), this conversion is nearly lossless (§4.1 validates this quantitatively). The analysis node taps activations from the stream before passing them to the SAE. For SAE training, this tap collects residual stream vectors at the split layer; for circuit tracing, the full activation stream across all layers can be preserved.

3.4 Distributed SAE Training

The distributed SAE trainer partitions the activation dataset across N workers. Each worker trains an independent SAE replica on its data partition. After each step, workers exchange parameter updates via weighted gradient averaging:

$$\theta_{\text{merged}} = \sum_{i=1}^N \alpha_i \theta_i, \quad \alpha_i = \frac{|\mathcal{D}_i|}{\sum_j |\mathcal{D}_j|} \quad (2)$$

For two equal-size partitions, $\alpha_0 = \alpha_1 = 0.5$. This is equivalent to synchronous data-parallel training when per-worker batch sizes are equal.

The SAE architecture is a TopK sparse autoencoder:

Listing 5: SAE forward pass. Decoder weight columns are unit-normalized.

```
def encode(x):
    pre_act = (x - b_dec) @ W_enc + b_enc
    features = ReLU(pre_act)
    topk_mask = topk_indices(features, k)      # zero all but top-k
    return features * topk_mask

def decode(features):
    return features @ W_dec + b_dec           # W_dec columns unit-norm

def loss(x):
    f = encode(x)
    x_hat = decode(f)
    return MSE(x, x_hat)
```

Dead feature detection monitors the number of features that have not fired over a rolling window of 5,000 tokens. Features with zero activations over this window are counted as “dead”—a metric that tracks dictionary utilization and provides an early signal of over-sparsification.

SAEs trained in this work. Four training runs were conducted (Table 1). The distributed training comparison used a 4,096-feature Llama-3.2-3B SAE. The cross-architecture matching analysis used a separate 16,384-feature Llama SAE trained single-node (see §4.4 for details).

3.5 Cross-Architecture Feature Matching

To identify features shared across model families, we use chunk-averaged activation pattern matching:

Table 1: SAEs trained in this work. All experiments used WikiText-103 as the source corpus.

Model	Layer	d_{in}	Dict	k	Steps	Corpus	Purpose
Llama-3.2-3B	14	3,072	4,096	64	50,000	500k	Dist. training comparison
Llama-3.2-3B	14	3,072	16,384	128	13,000 [†]	500k	Cross-arch matching
Mistral-7B [‡]	16	4,096	16,384	128	1,000	50k	Quality metrics + matching
Qwen2.5-3B	18	2,048	16,384	128	50,000	500k	Quality metrics + matching

[†] Training was ongoing; checkpoint at step 10,000 used for matching analysis. [‡] Activations extracted from a 4-bit quantized model (`mlx-community/Mistral-7B-v0.3-4bit`); bfloat16 variant requires HuggingFace authentication.

1. Define an evaluation corpus of 50,000 tokens.
2. Divide into $N_{\text{chunks}} = 100$ non-overlapping chunks of 500 tokens each.
3. For each model, collect SAE encoder outputs (post-TopK) for every chunk, then average over the token dimension to produce a per-chunk, per-feature activation vector. This yields a matrix $A \in \mathbb{R}^{N_{\text{chunks}} \times D}$, where D is the dictionary size.
4. Treat each column of A as a “fingerprint” of the corresponding feature over the evaluation corpus.
5. For a pair of models with matrices $A^{(1)}$ and $A^{(2)}$, compute pairwise cosine similarities:

$$\text{sim}(i, j) = \cos(A_{:,i}^{(1)}, A_{:,j}^{(2)}) \quad (3)$$

6. A match is declared when $\text{sim}(i, j) \geq \tau = 0.8$. For each feature in model 1, record the single highest-similarity match in model 2.
7. Universal features are those with matches in all three pairwise comparisons.

This method sidesteps the arbitrary choice of basis in each SAE’s decoder weight matrix and matches features by their functional behavior—the pattern of contexts they respond to. The matching analysis used the 16,384-feature SAEs for all three models (the single-node Llama SAE at step 10k, Mistral at step 1k, and Qwen at step 50k).

4 Results

4.1 Protocol Validation and Bandwidth

We validated the wire protocol using a two-process simulation of the Node A / Node B split on a single Mac Studio M2 Ultra, with model weights loaded independently in each process and activations communicated via Unix domain socket. The test ran a full inference pass of Llama-3.2-3B on a 512-token sequence split at layer 16 (layers 0–15 on Node A, layers 16–27 on Node B), using the actual model weights (`mlx-community/Llama-3.2-3B-bf16`).

Table 2: Split-inference correctness validation. Both the layer-15 handoff tensor and the layer-27 final output match the single-process baseline to machine zero.

Checkpoint	Tensor shape	max_abs_error	mean_abs_error	rms_error	Pass
Layer 15 output (handoff)	(512, 3072)	0.0	0.0	0.0	✓
Layer 27 output (final)	(512, 3072)	0.0	0.0	0.0	✓

Correctness (Table 2). The zero error reflects that bfloat16→float16 conversion is loss-less for activations in the observed range: float16’s 10-bit mantissa is wider than bfloat16’s 7-bit mantissa, so no precision is lost in this direction. The only rounding occurs on re-conversion at Node B (float16→bfloat16 before the next layer), which remains within float16 precision and produces no detectable error in the final output.

Throughput (Table 3). The localhost streaming benchmark used synthetic float16 activations in Llama-3.2-3B layer-16 shape (10,000 tokens, 128-token chunks over TCP loop-back).¹

Table 3: Localhost streaming throughput benchmark.

Metric	Value
Bytes transferred	61.4 MB
Messages sent / received	79 / 79
Elapsed time	0.049 s
Throughput	1.25 Gbps
Target	1.0 Gbps
Dropped messages	0
Sender/receiver byte count match	✓

At 1.25 Gbps, the protocol sustains throughput approximately **3,000× higher** than the real-time single-layer capture requirement for Llama-3.2-3B at 20 tokens/second (~ 120 KB/s).

Thunderbolt feasibility (Table 4). Required bandwidth at 20 tokens/second (full-depth, all-layer capture) versus available Thunderbolt 4 capacity (~ 3.5 GB/s practical):

Thunderbolt 4 is not a bottleneck under any realistic workload. The binding constraints are inference compute speed on Node A and SAE forward-pass throughput on Node B. Even at peak Mac Studio inference throughput (~ 300 tokens/second for the 3B model), TB4 utilization for full-depth capture stays below 1.4%.

¹Synthetic activations—fixed-seed random float16 in the correct shape—were used to isolate protocol performance from model inference time. The tested properties are protocol correctness and streaming throughput, not model output. The metadata field "weights": "synthetic-deterministic-seed42" in the benchmark results file records this.

Table 4: Required bandwidth vs. Thunderbolt 4 capacity at 20 tokens/second.

Model	Required MB/s	TB4 utilization	Margin
Llama-3.2-3B	3.3	0.09%	$\times 1,068$
Mistral-7B	5.0	0.14%	$\times 700$
Llama-2-13B	7.8	0.22%	$\times 448$
CodeLlama-34B	15.0	0.43%	$\times 233$

4.2 Distributed SAE Convergence

We compare distributed (2-worker) and single-node SAE training on Llama-3.2-3B activations (layer 14, 500k tokens, dict_size = 4,096, $k = 64$, batch = 256 tokens/worker = 512 total).

Table 5: Distributed vs. single-node SAE training at 5,000 steps (Llama-3.2-3B, layer 14).

Metric	Distributed	Single-node	Ratio
Initial loss	1.263	0.723	—
Final loss	0.01788	0.01778	1.005
Mean loss (all steps)	0.0608	0.0506	1.20
Converged?	✓	✓	—
Within 10% at step 5,000	✓	—	—

Proof-of-concept run, 5,000 steps (Table 5). The distributed run’s higher initial loss reflects a slower warmup (linear ramp over 200 steps vs. the baseline’s faster acceleration from better batch diversity), but both runs converge to essentially the same final loss (ratio 1.005). The mean loss over all steps is 20% higher for the distributed run due to this slower warmup phase, not slower final convergence.

Table 6: Distributed vs. single-node SAE training at 50,000 steps. The two-process setup was validated algebraically: gradient averaging over sequential workers is equivalent to synchronous data-parallel training.

Metric	Distributed	Single-node	Ratio
Final MSE	0.010613	0.010608	1.0005
Final L0	64.0	64.0	1.0
Final FVE	0.9822	0.9822	1.0
Dead features	1,109 (27.1%)	1,117 (27.3%)	—
Runtime	79.7 min	55.0 min	$1.45\times$
Verdict	PASS: within 0.05% MSE of single-node baseline		

Full run, 50,000 steps (Table 6). At 50,000 steps, the distributed SAE matches the single-node baseline to within 0.05% on MSE, 0% on L0 and FVE, and 0.2 percentage points

on dead feature rate. The distributed run is $1.45\times$ slower due to the gradient-averaging communication step; in a real two-node deployment over Thunderbolt, this overhead would be further reduced by overlapping gradient exchange with the next forward pass.

Convergence curves show that both runs follow the same trajectory after the warmup phase, with loss declining monotonically from ~ 0.02 at step 500 to ~ 0.0178 at step 5,000.

4.3 SAE Quality Metrics

Table 7 summarizes quality metrics for the SAEs used in the cross-architecture analysis. Fraction of Variance Explained (FVE) is the primary reconstruction quality metric; raw MSE is activation-scale-dependent and should not be compared across models with different hidden dimensions and activation norms.

Table 7: SAE quality metrics. FVE is the reconstruction quality metric comparable across models; MSE is activation-scale-dependent. Qwen’s higher MSE (relative to Llama) reflects larger activation norms at layer 18 of this model, not worse reconstruction quality.

Model	Layer	Dict	k	Steps	MSE	FVE	Dead features
Llama-3.2-3B (dist.)	14	4,096	64	50,000	0.01061	0.9822	1,109 (27.1%)
Mistral-7B [‡]	16	16,384	128	1,000	0.00315	0.9649	339 (2.1%)
Qwen2.5-3B	18	16,384	128	50,000	0.276	0.984	9,401 (57.4%)

[‡] 4-bit quantized model weights.

The Mistral SAE was trained on a smaller dataset (50k corpus tokens) and reached $\text{FVE} = 0.965$ in 1,000 steps. Dead features declined monotonically from 748 at step 1 to 339 at step 1,000. FVE oscillated between 0.896 and 0.944 during steps 100–900 (reflecting per-batch variance at this dataset scale), then reached 0.965 at step 1,000 as the cosine-annealing schedule drove the learning rate to its minimum.

The Qwen SAE shows 57.4% dead features at step 50,000—higher than the Llama (27.1%) and Mistral (2.1%) runs, though direct comparison is complicated by the $8\times$ expansion ratio ($d_{\text{in}} = 2,048 \rightarrow \text{dict} = 16,384$) and 204-epoch training regime. $\text{FVE} = 0.984$ confirms the SAE is capturing the representational structure of the layer despite the high dead-feature rate. Further hyperparameter tuning (lower expansion ratio, cosine annealing warmup) is expected to reduce the dead-feature rate.

The Llama distributed SAE’s $\text{FVE} = 0.9822$ at 27.1% dead features is consistent with published SAE results at this scale and suggests the 4,096-feature dictionary is slightly over-provisioned for a 3B model’s representational capacity at layer 14.

4.4 Cross-Architecture Feature Matching

The chunk-averaged activation matching analysis was run on all three pairwise combinations of the 16,384-feature SAEs (Llama single-node at step 10k, Mistral at step 1k, Qwen at step 50k), using 50,000 evaluation tokens from WikiText-103 divided into 100 chunks of 500 tokens each.

Table 8: Pairwise cross-architecture feature matches ($\cos \geq 0.8$). Coverage is the fraction of the 16,384-feature dictionary with at least one match in the paired model.

Pair	Matches	Coverage
Llama–Qwen	5,923	36.2%
Mistral–Qwen	8,099	49.5%
Llama–Mistral	12,174	74.3%
All three (universal)	3,753	22.9%

Pairwise match counts (Table 8).

Table 9: Venn diagram of features by overlap region. “Universal” features match all three pairwise comparisons at $\cos \geq 0.8$.

Region	Features
Llama only	11,155
Qwen only	13,784
Mistral only	8,087
Llama–Qwen (not Mistral)	414
Llama–Mistral (not Qwen)	1,261
Qwen–Mistral (not Llama)	307
All three (universal)	3,753

Feature distribution (Table 9). The 3,753 universal features are the central finding: a set of representational directions that, when measured by their activation pattern over a shared corpus, are essentially the same across Llama-3.2-3B, Mistral-7B, and Qwen2.5-3B. These three models share no training data (as far as is publicly known), differ in architecture (GQA vs. full attention, different MLP ratios), differ in scale (3B vs. 7B parameters), and were developed by different organizations. The convergence of their SAE features on a common set of $\sim 3,750$ directions is evidence that these directions correspond to features intrinsic to the training distribution rather than to any particular architectural choice.

The high Llama–Mistral match count (12,174; 74.3%) relative to the lower Llama–Qwen count (5,923; 36.2%) may reflect closer architectural lineage between Llama and Mistral (both derived from the LLaMA family) or differences in their training data distributions. The Mistral 4-bit quantization is a potential confound (§5.4).

Among the universal features, several reach cosine similarity 1.0 across all three pairwise comparisons—perfect alignment of activation patterns. These are the strongest candidates for features representing fundamental linguistic or semantic concepts that any large language model trained on English text would be expected to learn.

4.5 Safety Classifier Evaluation

We evaluated a feature-based safety classifier built on the Llama-3.2-3B SAE (layer 14, dict_size = 16,384, $k = 128$, trained 10,000 steps on WikiText-103). The classifier labels SAE features by semantic category using regex-based heuristic matching against maximum-activating token contexts, then aggregates “potentially-harmful” feature activations into a scalar harm score.

Table 10: Feature label distribution among the top-200 most frequently activated features on WikiText-103.

Category	Count	Share
factual	173	86.5%
stylistic	24	12.0%
code-related	2	1.0%
potentially-harmful	1	0.5%

Label distribution (top-200 features by activation frequency).

Table 11: Threshold sweep on the safety classifier. Random baseline F1 = 0.500 (balanced dataset).

Threshold	Precision	Recall	F1	FPR	Notes
0.10	0.500	1.000	0.667	1.000	Degenerate: flags everything
0.50	0.476	0.800	0.597	0.880	
0.70	0.488	0.630	0.550	0.660	
0.90	0.580	0.470	0.519	0.340	Best non-degenerate
1.10	0.732	0.300	0.426	0.110	
1.30	1.000	0.180	0.305	0.000	FPR = 0 but recall collapses

Classifier performance on 200-example balanced eval set (Table 11). The best non-degenerate operating point (threshold = 0.90) achieves F1 = 0.519, essentially indistinguishable from the 0.500 random baseline.

Root cause analysis. The failure is diagnostic, not accidental:

1. **Corpus mismatch.** WikiText-103 contains almost no explicitly harmful content. The top-200 features by activation frequency represent the model’s vocabulary for encyclopedic prose—factual, stylistic, code. Safety-relevant features are expected to be low-frequency on this corpus.
2. **Frequency bias.** Selecting features by activation frequency selects for generalist features, not discriminative ones. Safety-relevant features would rank in the bottom quartile of activation frequency on a Wikipedia corpus.

3. **Single-feature collapse.** The one identified harmful feature (feature 15040, a military-history detector firing on 9.9% of WikiText tokens) is too broad (fires on many safe military-history topics) and too narrow (captures weapons in a Wikipedia context, not harmful instructions in a chat context).
4. **Layer placement.** Layer 14 is at the midpoint of the 28-layer stack. Safety-relevant intent representations are more likely to emerge in later layers (20–28) where the residual stream has had time to integrate broader context.

Implications. The result motivates three specific changes: (a) replace the labeling corpus with a safety-targeted corpus such as AdvBench or HarmBench; (b) select features by *differential* activation frequency (harmful – safe) rather than absolute frequency; (c) target later layers. The harm scoring pipeline (two-pass frequency analysis, context replay, threshold sweep) is reusable—the bottleneck is input data quality, not pipeline logic.

5 Discussion

5.1 The Infrastructure Case

The primary contribution of this work is not any individual experimental result but the demonstration that the full interpretability stack—distributed inference, live activation streaming, online SAE training, cross-architecture feature analysis, and downstream safety evaluation—can run on two consumer nodes connected by a commodity interface. The alternative (cloud GPU clusters, specialized interconnects, large engineering teams) creates a high barrier to entry that concentrates interpretability research at a small number of well-funded organizations. Democratizing the infrastructure is a precondition for democratizing the science.

The Thunderbolt analysis (§4.1) establishes that the link is not a bottleneck under any foreseeable workload. The binding constraints are compute-side: inference speed on Node A and SAE forward-pass throughput on Node B. Future work should focus on these (larger batches, INT8 quantization for the SAE encoder) rather than the interconnect.

5.2 Universal Features and the Platonic Representation Hypothesis

The finding that 3,753 features match across all three model families at cosine similarity ≥ 0.8 is consistent with the Platonic Representation Hypothesis [7]: the conjecture that large neural networks trained on diverse data converge on a shared statistical model of reality, regardless of architecture or training procedure.

The universality is particularly striking given the architectural differences: Llama-3.2-3B uses grouped query attention with 3,072 hidden dimensions, Mistral-7B uses sliding window attention with 4,096 dimensions, and Qwen2.5-3B uses a different attention implementation with 2,048 dimensions. The universal features must therefore be aligned in *activation pattern space* (how they respond to text) rather than weight space (where they live in the parameter matrix). This is the correct notion of universality for interpretability: we care about what a feature responds to, not the weights that implement it.

Future work should test whether the universal features correspond to recognizable semantic or linguistic categories. The pattern matching method identifies which features align; human evaluation or automated labeling (e.g., by collecting maximum-activating tokens) would determine what they represent.

5.3 Safety Classifier: An Honest Failure

The safety classifier result is negative, and its honest framing is more valuable than a positive result would have been. A classifier achieving $F1 = 0.519$ on a balanced set, barely above chance, with a clean mechanistic explanation of why it fails, is a clearer contribution than a classifier achieving $F1 = 0.8$ on a benchmark without revealing its failure modes.

The mechanistic explanation is precise: the classifier fails because the features it uses were selected by a criterion (activation frequency on Wikipedia) orthogonal to the criterion relevant for the task (differential activation on harmful vs. safe text). This is not a subtle statistical failure—it is a pipeline design error fixable by changing the labeling corpus. The threshold sweep infrastructure, harm scoring formula, and evaluation dataset are all reusable. We propose that future iterations target $F1 \geq 0.80$ at $FPR \leq 0.10$, using features selected by contrastive activation analysis on a balanced corpus.

5.4 Limitations

Scale. All results are for models up to 7B parameters. The architecture supports 34B (§3), and the bandwidth analysis confirms feasibility, but we have not run 34B experiments. Larger models may exhibit qualitatively different universality patterns.

Corpus. All SAE training and evaluation used WikiText-103, an encyclopedic corpus that is clean and well-studied but is not representative of instruction-following or conversational distributions. SAEs trained on chat-format data may learn different features.

Quantization confound. The Mistral-7B activations were extracted from a 4-bit quantized model due to authentication constraints on the bf16 base model. Quantization reduces numerical precision of individual activations and may suppress some features (particularly those relying on fine-grained magnitude differences). The high Llama–Mistral match count (74.3%) suggests the key representational structure survives 4-bit quantization, but re-running with bf16 weights would confirm this.

Single split point. The current system uses a fixed split at one layer. Full-depth distributed capture (streaming all layers) is specified by the protocol but not demonstrated in these experiments. Full-depth capture would enable circuit-tracing experiments at scale.

Simulation vs. hardware. The two-node validation used two Python subprocesses on one physical Mac Studio rather than two physical machines. The protocol is hardware-agnostic, but the latency and jitter profile of a real inter-machine Thunderbolt link may differ from Unix domain socket performance.

5.5 Path to Production

The immediate next step is implementing the system in Swift/MLX rather than Python, using the same wire protocol. Swift provides access to macOS’s native socket APIs, lower-latency memory management, and better MLX integration for zero-copy buffer passing. The protocol spec is wire-format stable (v1.0.0); Swift and Python implementations are directly interoperable.

Beyond that, the natural extension is online SAE training: rather than collecting activations offline and training on stored data, the SAE trainer on Node B processes the activation stream as it arrives, updating weights continuously. This eliminates the two-stage collect→train pipeline and reduces the hyperparameter tuning cycle from hours to minutes.

The cross-architecture feature matching result also motivates a specific experiment: train a “universal SAE” whose dictionary is initialized from the matched features and fine-tuned jointly on activations from all three model families. If the universal features are truly shared representations, a single dictionary should achieve comparable FVE on all three models—a strong form of validation.

6 Conclusion

We have described MLXMESH, a system for distributed mechanistic interpretability on Apple Silicon hardware. The system streams intermediate layer activations between two nodes over a purpose-built binary wire protocol, enabling online SAE training and analysis without storing activations to disk. We have validated the system end-to-end—zero reconstruction error at the split boundary, 1.25 Gbps protocol throughput, distributed SAE training within 0.05% of the single-node baseline—and applied it to three experimental contributions: (1) a quantitative bandwidth analysis showing that Thunderbolt 4 is never a bottleneck at any model size considered; (2) cross-architecture feature matching identifying 3,753 universal features shared across Llama-3.2-3B, Mistral-7B, and Qwen2.5-3B; and (3) a safety classifier evaluation that honestly characterizes a negative result and diagnoses its precise failure mode.

The central message is that the infrastructure gap between “models small enough to study on one machine” and “models large enough to matter” is not a fundamental barrier—it is an engineering problem with a clean solution. Two Apple Silicon nodes, a Thunderbolt cable, and a well-specified wire protocol are sufficient for state-of-the-art interpretability research on models up to 34B parameters. The science of understanding what large language models represent does not require access to expensive centralized compute, and should not be limited to organizations that have it.

Acknowledgments

This work was conducted independently using consumer hardware. We thank the developers of MLX, the Hugging Face community for open model weights, and the mechanistic interpretability community whose open publication of methods and findings made this work possible.

Data Availability

All experimental data, training logs, benchmark results, and analysis scripts produced in this work are available at [https://github.com/\[author\]/mlxmesh](https://github.com/[author]/mlxmesh). This includes: activation collection scripts and metadata; SAE training checkpoints and training logs for all four runs; the cross-architecture matching results including the full list of 3,753 universal features; bandwidth benchmark results (clearly labeled as using synthetic activations); the distributed-capture validation JSON; and the safety classifier evaluation dataset and results. The mlxMesh wire protocol specification is documented in `docs/activation-streaming-protocol.md`.

References

- [1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [2] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv*, 2022. URL <https://arxiv.org/abs/2212.03827>.
- [3] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv*, 2023. URL <https://arxiv.org/abs/2309.08600>.
- [4] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.
- [5] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv*, 2024. URL <https://arxiv.org/abs/2406.04093>.
- [6] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [7] Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis. *arXiv*, 2024. URL <https://arxiv.org/abs/2405.07987>.

- [8] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.
- [9] Nikolaus Kriegeskorte, Marieke Mur, and Peter Bandettini. Representational similarity analysis — connecting the branches of systems neuroscience. *Frontiers in Systems Neuroscience*, 2:4, 2008. doi: 10.3389/neuro.06.004.2008.
- [10] Tom Lieberum, Senthooan Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Kramár, Neel Nanda, Caglar Sezener, Borja Balle, and Rohin Shah. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. *arXiv*, 2024. URL <https://arxiv.org/abs/2408.05147>.
- [11] Jack Lindsey, Amanda Gould, Adly Templeton, Callum McDougall, Joshua Batson, Adam Jermyn, Neil Thomas, Carson Denison, Amanda Askill, Anna Durbin, Trenton Bricken, Jason Marcus, Robert Lasenby, Tom Conerly, Tom Henighan, Nick Turner, Brian Chen, David Pearce, Joshua McDonald, Alex Tamkin, Aryan Guha, Aaron Jones, and Christopher Olah. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- [12] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [13] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2019.
- [14] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askill, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Christopher Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. URL <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>.
- [15] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Scheirer, Andy Jones, Andy Hyun, Alex Tamkin, David Bau, Jack Chen, and Christopher Olah. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- [16] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. *arXiv*, 2022. URL <https://arxiv.org/abs/2211.00593>.

- [17] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to AI transparency. *arXiv*, 2023. URL <https://arxiv.org/abs/2310.01405>.